

WeakMap and WeakSet - Simple Hinglish Summary

Source: <https://javascript.info/weakmap-weakset>

Main baat

Garbage collection chapter se yaad karo: JS engine value ko memory me tab tak rakhta hai jab tak wo **"reachable"** ho aur use ho sakti ho.

```
javascript

let john = { name: "John" };

// object access ho sakta hai, john uska reference hai

// reference overwrite karo
john = null;

// object memory se hat jayega
```

Aam taur par kisi object ki properties, ya array ya kisi aur data structure ke elements **tab tak reachable maane jate hain aur memory me rehte hain jab tak wo data structure memory me hai.**

Jaise object ko array me daalo, to jab tak array zinda hai, object bhi zinda rahega, chahe uska koi aur reference na ho:

```
javascript

let john = { name: "John" };

let array = [ john ];

john = null; // reference overwrite kiya

// john ka pehle wala object array ke andar stored hai
// isliye garbage-collect nahi hoga
// hum use array[0] se pa sakte hain
```

Isi tarah agar regular **Map** me object ko key banao, to jab tak **Map** hai, wo object bhi rahega. Wo memory leta hai aur garbage collect nahi ho sakta.

```
javascript
```

```
let john = { name: "John" };

let map = new Map();
map.set(john, "...");

john = null; // reference overwrite kiya

// john map ke andar stored hai,
// hum use map.keys() se pa sakte hain
```

WeakMap is baat me bilkul alag hai. Ye key objects ke garbage-collection ko nahi rokta.

1. WeakMap

Map aur **WeakMap** me **pehla farak**: **WeakMap** me **keys sirf objects** ho sakte hain, primitive values nahi:

```
javascript

let weakMap = new WeakMap();

let obj = {};

weakMap.set(obj, "ok"); // theek hai (object key)

// string ko key nahi bana sakte
weakMap.set("test", "Whoops"); // Error, kyunki "test" object nahi hai
```

Ab agar object ko key banao aur uska **koi aur reference na ho**, to wo **memory se (aur map se) apne aap hat jayega**.

```
javascript

let john = { name: "John" };

let weakMap = new WeakMap();
weakMap.set(john, "...");

john = null; // reference overwrite kiya

// john memory se hat gaya!
```

Upar ke regular **Map** example se compare karo. Ab agar **john** sirf **WeakMap** ki key ke roop me bacha hai, to wo map se (aur memory se) apne aap delete ho jayega.

`WeakMap` **iteration** aur `keys()`, `values()`, `entries()` **methods support nahi karta**, isliye isse saari keys ya values nikalne ka koi tarika nahi.

`WeakMap` ke sirf ye methods hain:

- `weakMap.set(key, value)`
- `weakMap.get(key)`
- `weakMap.delete(key)`
- `weakMap.has(key)`

Ye limitation kyu? Technical wajah se. Agar object ke saare references chale gaye (jaise upar `john`), to wo apne aap garbage-collect hona chahiye. Lekin **technically ye exactly tay nahi hai ki safai kab hogi**. JS engine decide karta hai. Wo turant safai kar sakta hai, ya ruk kar baad me jab zyada deletions hon. Isliye technically `WeakMap` ke abhi ke elements ki ginti pata nahi hoti. Engine ne safai ki ho ya na ki ho, ya aadhi ki ho. Isi wajah se saari keys/values tak pahunchne wale methods support nahi hain.

Ab aisa data structure kahan chahiye?

2. Use case: additional data

`WeakMap` ka main use **additional data storage** hai.

Agar kisi **dusre ke code** (shayad 3rd-party library) ke object ke saath kaam kar rahe ho aur uske saath kuch data store karna hai, jo **sirf tab tak rahe jab tak object zinda ho**, to `WeakMap` bilkul sahi hai.

Data `WeakMap` me daalo, object ko key banakar. Jab object garbage collect hoga, wo data bhi apne aap gayab ho jayega.

```
javascript
```

```
weakMap.set(john, "secret documents");  
// john mar gaya to secret documents apne aap destroy ho jayenge
```

Example: ek code users ka visit count rakhta hai. Jaankari map me hai: user object key, visit count value. Jab user chala jaye (uska object garbage collect ho jaye), to uska visit count rakhne ki zarurat nahi.

`Map` ke saath counting function:

```
javascript
```

```
// 📁 visitsCount.js
let visitsCountMap = new Map(); // map: user => visits count

// visits count badhao
function countUser(user) {
  let count = visitsCountMap.get(user) || 0;
  visitsCountMap.set(user, count + 1);
}
```

Aur code ka dusra hissa (shayad dusri file):

```
javascript

// 📁 main.js
let john = { name: "John" };

countUser(john); // uske visits gino

// baad me john chala jata hai
john = null;
```

Ab `john` object garbage collect hona chahiye, lekin wo **memory me rehta hai**, kyunki wo `visitsCountMap` ki key hai.

Users hatate waqt humein `visitsCountMap` saaf karna padega, warna wo memory me hamesha badhta rahega. Complex architectures me aisi safai **thakau kaam** ban sakti hai.

`WeakMap` se isse bacha ja sakta hai:

```
javascript

// 📁 visitsCount.js
let visitsCountMap = new WeakMap(); // weakmap: user => visits count

// visits count badhao
function countUser(user) {
  let count = visitsCountMap.get(user) || 0;
  visitsCountMap.set(user, count + 1);
}
```

Ab `visitsCountMap` saaf karne ki zarurat nahi. `john` object jab `WeakMap` ki key ke alawa har tarah se unreachable ho jata hai, to wo memory se hat jata hai, `WeakMap` me us key wali jaankari ke saath.

3. Use case: caching

Ek aur common example **caching** hai. Function ke results store ("cache") kar sakte hain, taaki same object par aage ke calls unhe reuse kar sakein.

Map se (optimal nahi):

```
javascript

// 📁 cache.js
let cache = new Map();

// result calculate karo aur yaad rakho
function process(obj) {
  if (!cache.has(obj)) {
    let result = /* obj ke liye result ki calculations */ obj;

    cache.set(obj, result);
    return result;
  }

  return cache.get(obj);
}

// Ab process() ko dusri file me use karte hain:

// 📁 main.js
let obj = { /* maan lo hamare paas ek object hai */ };

let result1 = process(obj); // calculate hua

// ...baad me, code ki kisi aur jagah se...
let result2 = process(obj); // cache se yaad rakha hua result mila

// ...baad me, jab object ki zarurat nahi rahi:
obj = null;

alert(cache.size); // 1 (Ouch! Object abhi bhi cache me hai, memory le raha hai)
```

Same object ke saath `process(obj)` ke kai calls me result sirf pehli baar calculate hota hai, phir `cache` se mil jata hai. **Nuksan:** jab object ki zarurat na rahe, to `cache` saaf karna padta hai.

`Map` ko `WeakMap` se replace karo to ye problem khatam ho jati hai. Object garbage collect hone ke baad cached result apne aap memory se hat jayega.

javascript

```
// 📁 cache.js
let cache = new WeakMap();

// result calculate karo aur yaad rakho
function process(obj) {
  if (!cache.has(obj)) {
    let result = /* obj ke liye result calculate karo */ obj;

    cache.set(obj, result);
    return result;
  }

  return cache.get(obj);
}

// 📁 main.js
let obj = { /* koi object */ };

let result1 = process(obj);
let result2 = process(obj);

// ...baad me, jab object ki zarurat nahi rahi:
obj = null;

// cache.size nahi mil sakta, kyunki ye WeakMap hai,
// lekin ye 0 hai ya jaldi hi 0 ho jayega
// obj garbage collect hone par cached data bhi hat jayega
```

4. WeakSet

WeakSet bhi isi tarah behave karta hai:

- **Set** jaisa hai, lekin **sirf objects** add kar sakte hain (primitives nahi)
- Object set me tab tak rehta hai jab tak wo **kahin aur se reachable** hai
- **Set** ki tarah **add**, **has** aur **delete** support karta hai, lekin **size**, **keys()** **nahi**, aur **koi iteration nahi**

"Weak" hone ki wajah se ye bhi additional storage ka kaam karta hai. Lekin arbitrary data ke liye nahi, balki "**haan/nahi**" **facts** ke liye. **WeakSet** me membership ka matlab object ke bare me kuch ho sakta hai.

Example: users ko **WeakSet** me daalkar track karo ki kaun humari site par aaya:

javascript

```
let visitedSet = new WeakSet();

let john = { name: "John" };
let pete = { name: "Pete" };
let mary = { name: "Mary" };

visitedSet.add(john); // John aaya
visitedSet.add(pete); // Phir Pete
visitedSet.add(john); // John dobara

// visitedSet me abhi 2 users hain

// kya John aaya?
alert(visitedSet.has(john)); // true

// kya Mary aayi?
alert(visitedSet.has(mary)); // false

john = null;

// visitedSet apne aap saaf ho jayega
```

`WeakMap` aur `WeakSet` ki sabse notable limitation: **iterations nahi aur abhi ka poora content nikalne ka koi tarika nahi**. Ye inconvenient lag sakta hai, lekin `WeakMap/WeakSet` ko apna main kaam (dusri jagah store/manage hone wale objects ke liye "additional" data storage) karne se nahi rokta.

Summary

`WeakMap` `Map` jaisa collection hai jo **sirf objects ko keys** ki tarah leta hai, aur jab wo kisi aur tarah se accessible nahi rehte to unhe associated value ke saath hata deta hai.

`WeakSet` `Set` jaisa collection hai jo **sirf objects** store karta hai, aur jab wo kisi aur tarah se accessible nahi rehte to unhe hata deta hai.

Mukhya fayda: inka objects ke liye **weak reference** hota hai, isliye garbage collector unhe aasani se hata sakta hai.

Keemat: `clear`, `size`, `keys`, `values`... ka support nahi milta.

`WeakMap` aur `WeakSet` "primary" object storage ke saath "secondary" data structures ki tarah use hote hain. Object primary storage se hat jaye aur sirf `WeakMap` ki key ya `WeakSet` me bacha ho, to wo apne aap saaf ho jata hai.

Practice Tasks (Answers)

1. "Unread" flags store karo: Messages ka array hai, lekin messages dusre ke code dwara manage hote hain (naye add, purane remove hote rehte hain, aur hume exact time pata nahi). Kaunsa data structure use karein jo batae ki message "read hua ya nahi"? Message array se hatne par hamare structure se bhi hat jana chahiye. Aur message objects me apni properties add nahi karni chahiye.

Jawab: `WeakSet`:

javascript

```
let messages = [
  {text: "Hello", from: "John"},
  {text: "How goes?", from: "John"},
  {text: "See you soon", from: "Alice"}
];

let readMessages = new WeakSet();

// do messages padhe gaye
readMessages.add(messages[0]);
readMessages.add(messages[1]);
// readMessages me 2 elements hain

// ...pehla message dobara padhte hain!
readMessages.add(messages[0]);
// readMessages me abhi bhi 2 unique elements hain

// jawab: kya message[0] padha gaya?
alert("Read message 0: " + readMessages.has(messages[0])); // true

messages.shift();
// ab readMessages me 1 element hai (technically memory baad me saaf ho sakti hai)
```

`WeakSet` messages ka set store karne aur kisi message ke hone ko aasani se check karne deta hai. Ye apne aap saaf ho jata hai. **Trade-off:** hum iterate nahi kar sakte, "saare padhe hue messages" seedha nahi le sakte. Lekin saare messages par iterate karke set me wale filter kar sakte hain.

Doosra tarika: message padhne ke baad `message.isRead = true` jaisi property add karna. Dusre ke code ke objects me aisa karna aam taur par discourage hota hai, lekin **symbolic property** se conflict bachaya ja sakta hai:

javascript

```
// symbolic property sirf hamare code ko pata hai
let isRead = Symbol("isRead");
messages[0][isRead] = true;
```

Symbols se problems ka chance kam hota hai, lekin **architecture ke hisaab se WeakSet** behtar hai.

2. Padhne ki dates store karo: Ab "haan/nahi" nahi, balki **date** store karni hai, aur wo sirf tab tak memory me rahe jab tak message garbage collect na ho.

Jawab: WeakMap:

```
javascript

let messages = [
  {text: "Hello", from: "John"},
  {text: "How goes?", from: "John"},
  {text: "See you soon", from: "Alice"}
];

let readMap = new WeakMap();

readMap.set(messages[0], new Date(2017, 1, 1));
// Date object hum baad me seekhenge
```

Quick Summary

Baat	Map / Set	WeakMap / WeakSet
Keys / values	Koi bhi type	Sirf objects
Garbage collection	Object ko rokte hain	Object ko nahi rokte (apne aap hatate hain)
Iteration	Hai	Nahi hai
size, keys(), clear()	Hai	Nahi hai
Methods	Bahut	WeakMap: set, get, delete, has WeakSet: add, has, delete

Kaam	Kya use karein
Object ke saath extra data (object ke saath hi khatam)	WeakMap
Caching (object ke saath hi khatam)	WeakMap
"Haan/nahi" fact object ke bare me	WeakSet
Dusre ke objects me properties add karni hon	WeakMap / WeakSet (properties add karne se behtar)

Yaad rakho: WeakMap/WeakSet hamesha "**secondary**" storage hain. Primary storage se object hat gaya to ye apne aap safai kar dete hain.